A10｜程序运行后，怎样观察和停止？

A09 关注用什么条件启动谁；A10 关注启动以后。**程序文件是代码，进程是一次运行实例**：同一文件能启动多次，停止实例不等于删除文件。

第一次先找回正文里的短程序：**一行输入 → 分别观察正常结果与诊断 → 程序结束 → 核对退出码。** 这条关系清楚后，再读后面的服务观察：查到自己的实例、找到监听入口、取得回应、停止并复查。两种程序共用"运行实例"的关系，不必一开始就记整组进程和端口命令。

输出、诊断和退出，怎样分开读？

`normalize_line.py` 把 `"  ticket-42  "` 去掉两端空格、转成大写。它交出 `TICKET-42`，另给一条"已去掉空格"的提示，最后退出 `0`。只输入空格时，没有有效编号，诊断指出"编号为空"，退出 `2`。这是正文程序的对照结果，不是当前电脑的实况。

当前问题	关系或短用法	记到什么程度
内容从哪里进、往哪里出？	`stdin` 标准输入；`stdout` 标准输出；`stderr` 诊断通道。默认 `print` 写 stdout，诊断可另写 stderr。	三个短名称值得认熟；屏幕同处显示，不代表同一通道。
stderr 有文字，就是失败吗？	诊断也可能只是提醒。程序结束后，还要按它的约定检查退出码与业务输出。	**有诊断 ≠ 非零退出；退出 0 也不替代内容核对。**
函数 return 0，就结束进程了吗？	`return` 先把值交回调用者；本例 `sys.exit(main())` 才把结果转成进程退出码。	区分代码内部返回与操作系统收到的结束结果。
这次退出码从哪里取？	当前 PowerShell 直接运行 Python 后，及时保存 `$LASTEXITCODE`；后续其他原生程序可能替换它。	认得读取入口及有效时机；后台任务、独立进程的结果不能拿当前 shell 的旧值代替。
想把结果和诊断分别存下？	`>` 把 stdout 交给文件，`2>` 把 stderr 交给文件；改变接收者，不改变程序算法。	认熟方向；**会覆盖已有目标**，先核对文件。完整重定向和编码差异查用。

回看：「一次短程序怎样交回结果」「有诊断文字，这次运行也可能成功」。本章的退出码 2 表示空编号；不同程序可以另作约定，不背一张通用数字含义表。

接下来改看会持续等待请求的本机文件服务。它不像短程序那样处理一行就结束，因此才需要以下查询、请求与停止证据；不是多记几条命令就能跳过对象核对。

查到程序、查到入口、取得回应，各证明什么?

当前问题	关系或短用法	记到什么程度
某个编号的进程还在吗?	`Get-Process -Id 4321` 查编号 4321 的进程。PID 是运行实例的编号，不是脚本名。	认熟 `Get-Process` 查进程、`-Id` 按编号；**数字按实际证据替换，不背示例 PID。**
名字是 python，就是自己那次吗?	还要核对解释器位置、启动命令中的脚本 / 服务目录，以及启动时间。同名和同编号都不足以永久确认身份。	记住"存在"和"属于本次"不同；详细 `Get-CimInstance` 查询查用。
服务在预期入口等连接吗?	`Get-NetTCPConnection` 查 TCP 连接 / 监听；监听行的 `LocalAddress`、`LocalPort` 是本地地址与端口，`OwningProcess` 是所属 PID。	**端口不是 PID。** 要把入口连接到自己的实例；完整筛选参数查用。
有入口，就取得正确结果了吗?	`Invoke-WebRequest` 发一次请求；`StatusCode` 是状态，`Content` 是正文。状态与预期内容一起看。	认熟请求与读结果；一次 `200` 不保证内容必然正确，更不保证以后永远可用。
该怎样停止并确认?	对自己前台运行的本章服务，用 `Ctrl+C` 请求结束，再查本次进程与监听。	**停止动作不等于停止证据。** 不同程序处理中断的方式可能不同。

一个短写法怎样读:

> 命令写法（示意）
>
> ```
> Get-Process -Id 4321
> ```

意思是"查询进程，按编号 4321 找"。下面只是缩减后的**示意输出**，不是当前电脑实况，也不要求运行这条示例:

> 输出示意
>
> ```
>  Id ProcessName
>  -- -----------
> 4321 python
> ```

`Id` 和 `ProcessName` 是结果的列名，不是命令。此结果只说明查询时有这个编号和名称，尚不能确认脚本身份、监听端口或服务内容。

地址 `127.0.0.1` 表示本机回环；示例端口 `8765` 是入口编号，不用背。`404` 可能是请求文件没找到，此时服务仍在运行；`200` 是 HTTP 状态，不是进程退出码。

回看: 「服务为什么一直占着当前终端」「从第二个终端追到真正的回应」「发出停止动作以后，还要看一次结果」。

停止前只认自己这次实例。 PID 会复用，不能拿旧编号去停止；先使用程序自己的结束方式。需要强制停止时，回看完整身份核对与收尾写法，不按名称结束所有 Python。查询命令报错不是"进程消失"；端口被另一程序接用，也不授权继续停止别人。

其余机制放在哪里，什么时候查？

当前问题	需要保留的关系	查用位置
管道交给下一工具什么？	PowerShell 命令间可传带属性的对象；跨到本例 Python 标准输入时，应约定文字、路径或 JSON，不直接变成 Python 对象。JSON 文字需解析后才按字段使用。	「管道里流动的，什么时候是文字，什么时候是对象」。不用背整套筛选脚本。
为什么另一个运行没拿到新配置？	子进程通常取得父进程启动当时环境的副本；之后改父进程不自动改已启动子进程，子进程改自己也不自动回写父进程。	「新进程从哪里得到环境变量」。保存 / 恢复旧值的配方可查。
想让提示符先回来，用什么？	`Start-Job` 建立会话管理的后台工作，job ID 不等于 Python PID；job 显示 `Completed` 也不证明里面 Python 成功。`Receive-Job` 取回输出。	「如果希望提示符先回来」。等待、读取内部退出码与清理配套查，不只复制启动那行。
独立启动后，谁查看与收尾？	`Start-Process` 启动实例，调用者须保留身份、日志位置和收尾责任；它不会自动增加健康检查或故障重启。	「独立启动以后，谁负责收尾」。对象的退出结果和句柄兼容写法查用。
能启动，为什么仍拒绝访问？	权限针对某个身份、目标和操作。`whoami` 查身份；能启动不等于能读写任意文件。	「遇到权限拒绝时，先定位是哪次操作」。先找具体失败点，不直接改系统权限或全部升管理员。

短程序先区分**正常结果、诊断和退出码**；换成持续服务后，再补上**进程存在不等于身份正确、入口监听不等于请求内容正确、停止动作以后仍需复查**。这些关系按当前对象找回，不是一整套等待背完的 PowerShell 词表。